

Systemdesign — Ameisenkolonie (Gruppe 11)

Reaktive Multi-Agenten-Simulation einer Ameisenkolonie auf einem 2D-Gitter. Single-threaded, deterministisch, in Python implementiert (`numpy`, `matplotlib`, `jsonschema`).

1. Komponenten

- **SimulationManager** — hält Welt und Agenten, fährt die Tick-Schleife, löst Konflikte (FCFS nach `agent_id`) und delegiert an das Logging. Einzige Komponente mit voller Weltsicht.
- **GridWorld** — Kapazitäts-Grid (0 = Hindernis). Hält `base_capacity`, `capacity` (durch dynamische Hindernisse zur Laufzeit mutiert), `occupancy`, `nests` und `food_sources`. 4er-Nachbarschaft.
- **PheromoneField** — drei `numpy`-Felder: `nest`, `food`, `neg`. `deposit_step` legt mit `deposit_initial` $\cdot \text{decay_per_step}^k$ ab, `evaporate` multipliziert pro Tick mit $(1-p)$, `zero_all` schaltet die Pheromone für einen Ausfall komplett ab.
- **AntAgent** — reaktiv, `sense` $\rightarrow$ `decide` $\rightarrow$ `act`. Zustand: `position`, `energy`, `carrying`, `step_k` (Schritte seit Nest/Quelle), `path` (Pfadgedächtnis der letzten 32 Zellen, gegen Zyklen) und `neg_active`: bleibt eine Ameise auf der Futtersuche an einer Zelle mit hohem Food-Pheromon stehen, ohne dort tatsächlich Futter zu finden, legt sie auf den folgenden Schritten Negativ-Pheromon auf bereits verlassenen Zellen ab, bis sie Futter findet oder zum Nest zurückkehrt.
- **JsonLogger** — schreibt `events.jsonl` (eine Zeile pro Tick mit Aggregatmetriken + per-Quelle), Snapshots als `.npz` (alle Pheromonkanäle)
- Kapazität zum Snapshot-Zeitpunkt, damit Heatmaps dynamisch-blockierte Felder korrekt maskieren) und `summary.json` am Lauf-Ende.

2. Tick-Ablauf

1. **Events** — geplante dynamische Hindernisse erscheinen/verschwinden; jede Ameise, die unter einem auftauchenden Hindernis steht, stirbt sofort.
2. **apply_actions** — Aktionen aus dem Vortick FCFS ausführen (Move/Pickup/Drop/NoOp), dabei Pheromone deponieren. Beim Betreten einer Nest- oder Futterzelle wird die Energie implizit auf `energy_max` aufgefrischt.
3. **sense $\rightarrow$ decide** — jede lebende Ameise zieht 1 Energie ab, nimmt Items + Pheromone der Nachbarschaft wahr und legt ihre nächste `ActionRequest` in die Manager-Queue.
4. **Pheromone** — `evaporate()`, oder `zero_all()` während eines Ausfalls.
5. **Logging** — Tick-Zeile, ggf. Snapshot.

3. Entscheidungsregel

Pro Ameise: mit Wahrscheinlichkeit ϵ zufällige laufbare Nachbarzelle (ϵ -Greedy-Exploration), sonst Softmax über $\text{score} = w_{\text{food}} \cdot \text{food} - \text{neg_weight} \cdot \text{neg}$ auf Futtersuche bzw. $\text{score} = w_{\text{nest}} \cdot \text{nest} - \text{neg_weight} \cdot \text{neg}$ auf dem Rückweg. Zellen aus dem Pfadgedächtnis werden bestraft, damit Ameisen nicht zwischen denselben Feldern oszillieren.

4. Konfiguration und Auswertung

Jedes Experiment liegt als JSON-Datei vor, validiert gegen `world.schema.json`. Im Schema enthalten sind unter anderem `obstacles`, `dynamic_obstacles` (Rechtecke mit `appears_at`/`disappears_at`), `warm_start.trails` und `warm_start.lines` (vorinstallierte Pheromonpfade über Bresenham-Linien) sowie `perturbations.pheromone_outage` für den Pheromon-Ausfall. Pro Lauf landet alles in `runs/<run_id>/` (`header.json`, `events.jsonl`, `capacity.npy`, `snapshots/tick_*.npz`, `summary.json`). `plot.py` erzeugt daraus `cumulative.png`, `rate.png`, `t_first.png` und Pheromon-Heatmaps (2x3 Snapshots pro Kanal).